CANE WALLET

Bender Alawwad, Blaise Driscoll, Preston Gmernicki

CSC 431 - UNIVERSITY OF MIAMI

Agenda

01 System Analysis

- Overview
- Diagram
- Actor Identification

02 Design Rationale

- Architectural Style
- Design Pattern
- Framework

03 Functional Design

- Classes
- Class Diagram

01

System Analysis

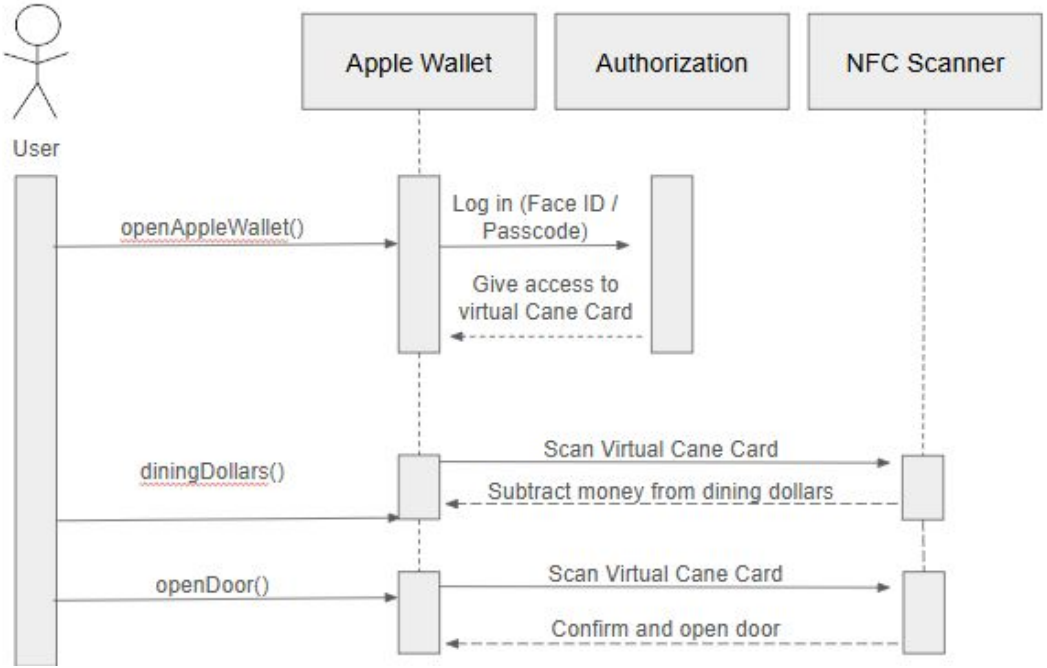
System Overview

Cane Wallet is a digital ID solution for university students, allowing them to add their Cane Card to Apple Wallet.

It enables payment with Dining Dollars, access to campus buildings, and resource checkout using NFC technology.

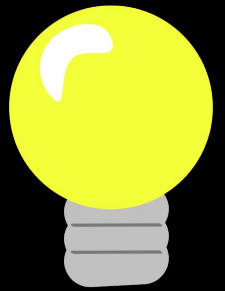
The architecture emphasizes modularity and Apple Wallet API integration, designed exclusively for iOS platforms.

Sequence Diagram



Actor Identification

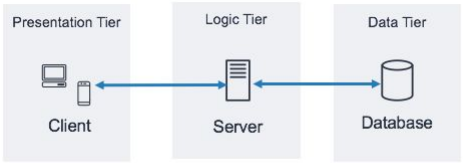
- **Student**: Adds Cane Card, makes payments, accesses buildings, checks out resources.
- **University System**: Authenticates and manages student records.
- **Apple Wallet**: Stores digital Cane Card and interacts via NFC.
- **NFC Terminal**: Accepts inputs from Apple Wallet and grants access or completes transactions.
- **Campus Facilities:** Backend systems for dining, access control, and library/resource tracking.



02

Design Rationale

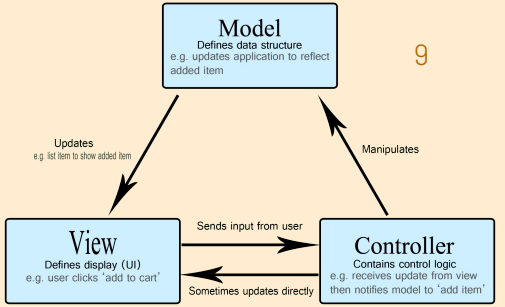
Architectural Style



3-Tier Architecture:

- Presentation Tier: iOS front-end app + Apple Wallet interface
- Logic Tier: University backend authentication and control systems
- Data Tier: Student and transaction data storage systems

Design Pattern



Model-View-Controller (MVC): Provides a clear separation of concerns, which is ideal for managing a modular and scalable iOS application.

- The Model (data and business logic),
- The View (user interface through Apple Wallet and app screens),
- The Controller (logic handling user interactions and system responses).

Easier to maintain and scale each component independently.
Ex: changes to the Apple Wallet UI do not impact the transaction processing logic.

Model

The Model layer is the foundation of the Cane Wallet system, responsible for managing data, business rules, and application state. It operates independently of the user interface and provides data to the Controller upon request.

Ex: Student, CaneCard, Transaction, AccessControl, AuthenticationManager.

View

This is the interface the user interacts with — typically a front-end app or Apple Wallet screen

- The Apple Wallet interface that shows the Cane Card, balance, and prompts for NFC interaction.
- Any in-app views showing login forms, wallet linking confirmation, or transaction status.

Controller

The Controller receives input from the user (e.g., tap phone to NFC reader), processes it by interacting with the Model, and then updates the View.

Responsibilities in the Controller:

- Receives user actions (e.g., tap to pay).
- Calls methods like Student.makeTransaction() or AccessControl.validateAccess(...).
- Updates the UI with feedback (success/failure, updated balance, error messages).

Framework

Apple Wallet API: Facilitates seamless integration with digital cards.
Swift: Primary language due to iOS exclusivity.
Xcode: Integration Development Environment for building and testing iOS applications.

These frameworks are designed for Apple products, ensuring best-in-class performance and security, reducing development needs with pre-built, secure functionality that aligns with the system's requirements.

03

Functional Design

Classes

- **Student**: Stores user info like name, ID, device link status. Contains methods like authenticate() and makeTransaction().
- **CaneCard**: Holds data about the digital card (balance, access permissions). Handles updateBalance() and verifyAccess().
- **Transaction**: Manages payment records and logs via processTransaction() and logTransaction().
- **AccessControl**: Handles logic related to determining whether a student can access a specific building.
- **AuthenticationManager**: Handles security-related logic like issuing and verifying tokens.
- **NFCScanner:** Reads the Cane Card from apple wallet and transmits the data

Class Diagram